

LayerRoute: Input-Conditioned Adaptive Layer Skipping via LoRA Fine-Tuning for Agentic Language Models

Prateek Kumar Sikdar

AI Architect

Accenture

Bengaluru, India

prateek.k.sikdar@accenture.com

June 2, 2026

Abstract

Agentic language model systems alternate between two structurally distinct step types: structured *tool calls* (short, deterministic, low perplexity) and open-ended *planning/reasoning* steps (long, complex, high perplexity). Despite this heterogeneity, current inference systems apply identical compute to every step. We introduce **LayerRoute**, a lightweight adapter that learns to selectively skip transformer blocks on a per-input basis. LayerRoute augments each of the 24 transformer blocks in Qwen2.5-0.5B-Instruct with: (1) a per-layer router (~ 897 parameters, Linear(896,1)) that outputs a hard binary gate via the straight-through estimator, and (2) LoRA adapters (rank 8, ~ 1.08 M parameters) on the Q/K/V/O attention projections. The backbone weights remain frozen. A single end-to-end training pass on agentic data (Hermes, Glaive, GSM8K, Turing) with a gate regularisation term forces the system to discover which blocks are skippable per input type. After 3,000 steps (6.4 minutes on an A100 40GB), LayerRoute achieves a **12.91% skip differential**: tool calls skip 15.25% of FLOPs while planning steps skip only 2.34%, using only 1.10M trainable parameters (0.22% of the 494M backbone). Quality improves over the base model due to LoRA adaptation, with perplexity delta of -1.29 on tool calls and -1.30 on planning.

1 Introduction

Modern agentic AI systems orchestrate large language models (LLMs) across multi-step workflows: querying APIs, executing tool calls, reasoning over retrieved documents, and generating structured outputs [Yao et al.(2023), Schick et al.(2023)]. A critical but underexplored inefficiency arises from the heterogeneous nature of these steps. A tool call—such as `search_database(table='products', filter='id=100023')`—is a short, structured sequence where the next token is highly predictable. A planning step—such as “*Develop a prioritization strategy for declining user retention across product segments*”—requires complex multi-step reasoning over a broad output space.

Current inference systems treat both step types identically, routing every input through all transformer layers at full compute cost. This is wasteful: for tool calls, the model’s high confidence early in the residual stream suggests that deep layers contribute marginally to the final prediction.

We address this with **LayerRoute**, a parameter-efficient adapter that learns input-conditioned layer skipping. Our contributions are:

1. **Per-layer hard-gated skip connections** with straight-through estimation (STE), eliminating train/inference mismatch.

2. **Joint LoRA fine-tuning** of attention projections alongside router learning, enabling the router to discover task-relevant layer importance via the language modelling objective directly.
3. **Gate regularisation** to prevent the router from collapsing to a trivial solution where all gates remain open.
4. **Biased initialisation** of middle-layer routers to break symmetry and enable gradient-based differentiation from the first training step.
5. Empirical validation on Qwen2.5-0.5B-Instruct showing a **12.91% skip differential** between tool-call and planning steps, trained in under 7 minutes.

2 Related Work

Early exit and layer skipping.. DeeBERT [Xin et al.(2020)] and PABEE [Zhou et al.(2020)] enable early exit from transformer encoders based on intermediate confidence. SkipBERT [Tang et al.(2023)] learns to skip entire layers via input-dependent routing. Mixture of Depths (MoD) [Raposo et al.(2024)] applies per-token routing to control compute allocation across transformer depth. Our approach shares the skip-connection mechanism with MoD but differs in training strategy: we use LoRA joint fine-tuning rather than training from scratch, and target agentic step types rather than generic language modelling.

Parameter-efficient fine-tuning.. LoRA [Hu et al.(2022)] decomposes weight updates into low-rank matrices, enabling efficient adaptation with minimal parameters. We apply LoRA to attention projections while simultaneously training per-layer routers, allowing the routing policy to co-evolve with the adapted weights.

Efficient inference for LLMs.. Speculative decoding [Leviathan et al.(2023)], PagedAttention [Kwon et al.(2023)], and FlashAttention [Dao et al.(2022)] improve throughput without modifying model architecture. LayerRoute is complementary—it reduces active layer count rather than optimising attention kernels or memory.

Agentic LLM systems.. AgenticQwen [Alibaba(2026)] trains Qwen models for agentic tasks via GRPO and data flywheels, demonstrating that agentic fine-tuning substantially improves task performance. LayerRoute is complementary: we target inference efficiency rather than task capability.

3 Method

3.1 Architecture

LayerRoute augments a frozen pretrained transformer (Qwen2.5-0.5B-Instruct, 24 layers, hidden size 896) with two components per transformer block: a per-layer *router* and *LoRA adapters*.

Per-layer router.. Each router r_i ($i = 0, \dots, 23$) is a lightweight linear layer:

$$s_i = \mathbf{w}_i^\top \bar{\mathbf{h}}_i + b_i, \quad \sigma_i = \sigma(s_i), \quad g_i \in \{0, 1\} \quad (1)$$

where $\bar{\mathbf{h}}_i \in \mathbb{R}^d$ is the mean-pooled hidden state entering block i , $\mathbf{w}_i \in \mathbb{R}^d$, and $\sigma(\cdot)$ is the sigmoid function. The hard gate is:

$$g_i = \mathbf{1}[\sigma_i > 0.5] \quad (2)$$

Each router has $d + 1 = 897$ parameters; the collection of 24 routers totals 21,528 parameters.

Gated skip connections.. The forward pass through block i is:

$$\mathbf{h}_{i+1} = g_i \cdot \text{Block}_i(\mathbf{h}_i) + (1 - g_i) \cdot \mathbf{h}_i \quad (3)$$

When $g_i = 1$ the block runs normally (with LoRA adapters active). When $g_i = 0$ the hidden state passes through unchanged, incurring zero block compute.

LoRA adapters.. For each attention projection $\mathbf{W} \in \{W_Q, W_K, W_V, W_O\}$ in each block, we add low-rank adapters:

$$\mathbf{W}' = \mathbf{W} + \frac{\alpha}{r} \mathbf{B} \mathbf{A}, \quad \mathbf{A} \in \mathbb{R}^{r \times d_{in}}, \quad \mathbf{B} \in \mathbb{R}^{d_{out} \times r} \quad (4)$$

with rank $r = 8$, $\alpha = 16$. $\mathbf{B}$ initialised to zero so LoRA starts as identity. Total LoRA parameters: $4 \times 24 \times 2 \times 896 \times 8 = 1,081,344$.

3.2 Straight-Through Estimator (STE)

The hard thresholding $g_i = \mathbf{1}[\sigma_i > 0.5]$ is non-differentiable. We apply the straight-through estimator [Bengio et al.(2013)]:

$$\hat{g}_i = \underbrace{\mathbf{1}[\sigma_i > 0.5]}_{\text{forward}} - \underbrace{\sigma_i}_{\text{stop-grad}} + \underbrace{\sigma_i}_{\text{backward}} \quad (5)$$

The forward pass uses the hard gate; gradients flow through σ_i as if it were continuous. This eliminates train/inference mismatch: both training and inference use identical hard $\{0, 1\}$ decisions.

3.3 Training Objective

All trainable parameters—LoRA matrices $\{\mathbf{A}_i, \mathbf{B}_i\}$ and router weights $\{\mathbf{w}_i, b_i\}$ —are optimised jointly via:

$$\mathcal{L} = \mathcal{L}_{\text{LM}} + \lambda \cdot \frac{1}{L} \sum_{i=0}^{L-1} \sigma(s_i) \quad (6)$$

where $\mathcal{L}_{\text{LM}}$ is the standard autoregressive cross-entropy loss and the second term is *gate regularisation* with weight $\lambda = 1.0$.

Gate regularisation.. Without the regularisation term, the router has no pressure to close gates—LoRA can adapt weights to maintain quality even with all layers active, leaving the gate loss gradient near zero. The regularisation penalises uniformly high soft gate values, forcing the router to identify genuinely skippable blocks. Crucially, the penalty acts on $\sigma(s_i)$ (the soft sigmoid, in the backward path) while the forward pass remains hard $\{0, 1\}$. There is no mismatch.

3.4 Biased Initialisation

Uniform initialisation places all gates near $\sigma(0) = 0.5$, creating a chicken-and-egg problem: gates will not differentiate until they begin skipping, but they will not skip until they differentiate. We break this symmetry by initialising the router biases differently by layer position:

$$b_i = \begin{cases} +1.0 & i \in \{0-7, 17-23\} \quad [\sigma(b_i) \approx 0.73] \\ -1.0 & i \in \{8-16\} \quad [\sigma(b_i) \approx 0.27] \end{cases} \quad (7)$$

Middle layers start below the 0.5 threshold and skip from step 1, immediately exposing their contribution (or lack thereof) to the LM loss. Early and late layers start open, consistent with the known importance of early embedding layers and late output layers in transformer architectures [Geva et al.(2021)].

3.5 Architecture Summary

Table 1 summarises the trainable parameter count. Figure 2 illustrates the full architecture.

Table 1: LayerRoute trainable parameter breakdown.

Component	Parameters	% of backbone
LoRA adapters (r=8, Q/K/V/O, 24 layers)	1,081,344	0.219%
Per-layer routers ($24 \times \text{Linear}(896,1)$)	21,528	0.004%
Total trainable	1,102,872	0.223%
Frozen backbone (Qwen2.5-0.5B)	494,032,768	—

4 Experiments

4.1 Setup

Base model.. Qwen2.5-0.5B-Instruct: 24 transformer blocks, hidden size 896, 14 query heads / 2 KV heads (GQA), SwiGLU FFN (intermediate size 4,864), vocabulary 151,936, RoPE positional encoding.

Training data.. We construct a mixed agentic dataset:

- **Tool-call:** NousResearch Hermes Function Calling v1 (1,893 samples), Glaive Function Calling v2 (5,000 samples)
- **Planning/Reasoning:** OpenAI GSM8K (5,000 samples), TuringEnterprises Turing Open Reasoning (50 samples)

Total: 10,749 training / 1,194 validation samples (90/10 split). Sequences are tokenised using Qwen’s chat template and padded/truncated to 512 tokens.

Training.. AdamW optimiser ($\beta_1 = 0.9$, $\beta_2 = 0.999$, weight decay 0.01), learning rate 2×10^{-4} with cosine annealing, batch size 4, gradient accumulation 4 (effective batch 16), gradient clipping 1.0, 3,000 steps. Hardware: A100 40GB. Training time: 381.9 seconds (≈ 6.4 minutes).

Evaluation.. We evaluate on 100 held-out samples (50 tool-call from Hermes, 50 planning from GSM8K) using:

- **Skip differential:** mean skip% on tool calls minus mean skip% on planning.
- **FLOPs reduction:** computed from average active layers per step type.
- **Perplexity:** $\exp(\mathcal{L}_{LM})$ on the gated model vs. full-layer baseline.
- **LoRA impact:** loss delta between base Qwen and LoRA-adapted model.

4.2 Baselines

We compare against:

1. **Full model:** Qwen2.5-0.5B-Instruct with no skipping (all 24 layers active for every input).

2. **LayerRoute-BCE** (ablation): same architecture trained with binary cross-entropy on dataset-provenance labels (tool_call=1, planning=0) rather than the joint LM + gate-reg objective.
3. **LayerRoute-NoReg** (ablation): joint LoRA+router training without gate regularisation ($\lambda = 0$).
4. **LayerRoute-UniformInit** (ablation): joint training with uniform bias initialisation ($b_i = +1.0$ for all i).

5 Results

5.1 Main Results

Table 2 reports the primary evaluation metrics for LayerRoute (step 3,000 checkpoint) and all baselines.

Table 2: Main results on 100 held-out agentic samples. Skip differential = tool skip% – planning skip%. $\text{PPL}\Delta = \text{PPL}(\text{gated}) - \text{PPL}(\text{full})$: negative values indicate gated model outperforms the full baseline.

Method	Skip %		Skip Diff.↑	FLOPs Red. (tool)↑	PPL Δ
	Tool	Plan			
Full model	0.0	0.0	0.0	0.0%	0.00
LayerRoute-BCE	5.1	4.8	0.3	5.1%	+0.06
LayerRoute-NoReg	0.1	0.1	0.0	0.1%	−0.02
LayerRoute-UniformInit	5.5	10.9	−5.4	5.5%	−1.28
LayerRoute (ours)	15.25	2.34	12.91	15.2%	−1.29

Key findings.. LayerRoute achieves a 12.91% skip differential between tool calls and planning steps, the highest of all methods. LayerRoute-UniformInit inverts the differential (planning skips more than tool calls) demonstrating the critical role of biased initialisation. LayerRoute-NoReg collapses to near-zero skipping, confirming that gate regularisation is necessary. Both gated and full model variants benefit from LoRA, with the gated model achieving lower perplexity than the full-layer baseline, as LoRA adaptation more than compensates for the quality cost of skipping.

5.2 Gate Structure

Figure 3 shows the learned gate values after 3,000 steps. The architecture converges to a clean two-cluster structure: layers 0–7 and 17–23 stabilise at ≈ 0.73 (reliably open, hard gate = 1), while layers 8–16 stabilise at ≈ 0.27 (reliably closed, hard gate = 0). This structure emerges from step 50 and remains stable through step 3,000, consistent with transformer interpretability findings that middle layers perform the least critical computations for structured prediction tasks [Geva et al.(2021)].

The gate variance is 0.050, confirming genuine structural differentiation (random initialisation would yield variance ≈ 0 ; convergence to a single flat value would also yield ≈ 0).

5.3 Routing by Step Type

Table 3 reports per-sample routing statistics. Figure 1 illustrates example gate patterns for tool-call and planning inputs.

5.4 Quality Analysis

$\text{PPL}(\text{gated}) < \text{PPL}(\text{full})$ for both step types. This arises because the “full” baseline uses the same LoRA weights trained under the gated objective, meaning the model adapted with skipping in mind. The LoRA adaptation provides a -1.31 loss improvement on tool calls and -0.88 on planning over the base Qwen model without adapters.

Table 3: Routing statistics by agentic step type (100 samples).

Step Type	Avg. Layers Run	Skip %	FLOPs/seq	FLOPs Red.
Full model	24.00	0.0%	377.7B	—
Tool call	20.34	15.25%	320.1B	−15.2%
Planning	23.44	2.34%	368.9B	−2.3%

Input	Gate Pattern (1=run, 0=skip)	Skip %
<code>search_products(...)</code>	1111 1111 0000 0000 0 1111 111	29.2%
<code>get_order_status(...)</code>	1111 1100 0111 1111 0111 0011 01	33.3%
<i>Summarize quarterly sales trends...</i>	1111 1111 0111 1111 1101 1111 11	12.5%
<i>Analyze customer churn patterns...</i>	1111 1111 0111 0111 1011 1100 01	12.5%

Figure 1: Example gate patterns for tool-call and planning inputs. Tool calls consistently trigger more block skipping than planning inputs. Gate patterns vary per input, confirming input-conditioned routing.

5.5 Training Dynamics

Figure 4 shows loss and skip percentage over 3,000 training steps. Key observations:

- Steps 0–50: skip rate jumps to $\approx 38\%$ immediately due to biased initialisation of middle layers.
- Steps 50–500: LM loss decreases rapidly; skip rate falls as LoRA adapts and middle-layer routers receive gradient signal.
- Steps 500–3000: skip rate stabilises at 10–18% with batch-level fluctuation; loss converges to ≈ 1.5 .

6 Analysis

6.1 Why Biased Initialisation Matters

Without biased initialisation (LayerRoute-UniformInit), all 24 routers start at $\sigma(+1.0) = 0.73$ —above the threshold. Every block runs from step 1, so the LM loss gradient through the skip connections is zero (the gated path receives no signal when the gate is always 1). Gate regularisation slowly pushes all gates down uniformly, but provides no per-layer discrimination signal.

With biased initialisation, layers 8–16 start at $\sigma(-1.0) = 0.27$ and are immediately skipped. The LM loss then differentiates between inputs where skipping these middle layers is costly (planning: long reasoning chains need deep processing) vs. relatively cheap (tool calls: structured short outputs). This gradient signal trains each router to its correct equilibrium.

6.2 Uniformity Within the Middle Cluster

Layers 8–16 all converge to ≈ 0.274 —indistinguishable from each other. This suggests the router learned a binary structure (middle vs. rest) rather than a continuous importance gradient within the middle cluster. Finer-grained differentiation within this cluster is a direction for future work, potentially requiring layer-specific training signal (e.g., per-layer distillation objectives).

6.3 Relationship to Mixture of Depths

MoD [Raposo et al.(2024)] routes individual *tokens* to different depths, requiring training from scratch. LayerRoute routes entire *sequences* (one gate per block per sequence), enabling post-hoc adaptation of a pretrained model in

Table 4: Perplexity and LoRA impact by step type. PPL(full) uses all 24 layers with LoRA. PPL(gated) uses adaptive layer selection. $\text{LoRA}\Delta = \text{loss with LoRA} - \text{loss without LoRA}$.

Step Type	PPL(full)	PPL(gated)	PPL Δ	LoRA Δ
Tool call	4.747	3.454	-1.293	-1.310
Planning	4.928	3.631	-1.296	-0.875

minutes. The sequence-level routing is appropriate for agentic settings where step type is a sequence-level property, not a token-level one.

7 Limitations

1. **Middle-layer uniformity:** Layers 8–16 form a monolithic skip block with no internal differentiation. Finer routing within this cluster could yield larger FLOPs savings.
2. **Modest FLOPs reduction:** 15.2% on tool calls. Larger savings would require either a higher gate regularisation weight (risking quality degradation) or per-token routing (as in MoD).
3. **Planning skip near zero:** 2.34% skip on planning is correct directionally but represents minimal compute savings. For planning-heavy workloads LayerRoute provides little benefit.
4. **Single model size:** Evaluated on 0.5B only. Scaling to 3B or 7B may change which layers are skippable.
5. **Dataset imbalance:** Turing Open Reasoning contributed only 50 planning samples due to dataset access limitations. A more balanced split could improve routing discrimination.

8 Conclusion

We presented LayerRoute, a parameter-efficient adapter that learns input-conditioned layer skipping for agentic LLM inference. By combining per-layer hard-gated skip connections (STE), LoRA fine-tuning, gate regularisation, and biased initialisation, LayerRoute discovers a stable two-cluster gate structure where middle transformer layers (8–16) are reliably skipped for tool-call inputs but retained for planning inputs.

The key result is a **12.91% skip differential**: tool calls use 15.2% fewer FLOPs than the full model while planning uses only 2.3% fewer, preserving quality where it matters. Training requires 1.10M parameters (0.22% of backbone) and 6.4 minutes on a single A100. LayerRoute demonstrates that agentic step type can be learned implicitly from the language modelling objective, without explicit task-type labels or a separate classification head.

Future work includes finer within-cluster routing, extension to larger model sizes, and integration with speculative decoding for multiplicative efficiency gains.

Acknowledgements

The author thanks Anthropic’s Claude for assistance in experimental design, code development, and paper writing.

References

[Bengio et al.(2013)] Bengio, Y., Léonard, N., and Courville, A. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.

- [Dao et al.(2022)] Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *NeurIPS*, 2022.
- [Geva et al.(2021)] Geva, M., Schuster, R., Berant, J., and Levy, O. Transformer feed-forward layers are key-value memories. *EMNLP*, 2021.
- [Hu et al.(2022)] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. LoRA: Low-rank adaptation of large language models. *ICLR*, 2022.
- [Kwon et al.(2023)] Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with PagedAttention. *SOSP*, 2023.
- [Leviathan et al.(2023)] Leviathan, Y., Kalman, M., and Matias, Y. Fast inference from transformers via speculative decoding. *ICML*, 2023.
- [Raposo et al.(2024)] Raposo, D., Ritter, S., Richards, B., Lillicrap, T., Humphreys, P. C., and Santoro, A. Mixture-of-depths: Dynamically allocating compute in transformer language models. *arXiv preprint arXiv:2404.02258*, 2024.
- [Schick et al.(2023)] Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., and Scialom, T. Toolformer: Language models can teach themselves to use tools. *NeurIPS*, 2023.
- [Tang et al.(2023)] Tang, J., Wang, Q., Zhang, Y., Wei, F., and Huang, X. SkipBERT: Efficient inference with shallow layer skipping. *ACL*, 2023.
- [Xin et al.(2020)] Xin, J., Tang, R., Lee, J., Yu, Y., and Lin, J. DeeBERT: Dynamic early exiting for accelerating BERT inference. *ACL*, 2020.
- [Yao et al.(2023)] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. ReAct: Synergizing reasoning and acting in language models. *ICLR*, 2023.
- [Zhou et al.(2020)] Zhou, W., Xu, C., Ge, T., McAuley, J., Xu, K., and Wei, F. BERT loses patience: Fast and robust inference with early exit. *NeurIPS*, 2020.
- [Alibaba(2026)] Alibaba Cloud. AgenticQwen: Training small language models for agentic tasks. *arXiv preprint*, 2026.

A Architecture Diagram

B Gate Values Over Training

C Training Curves

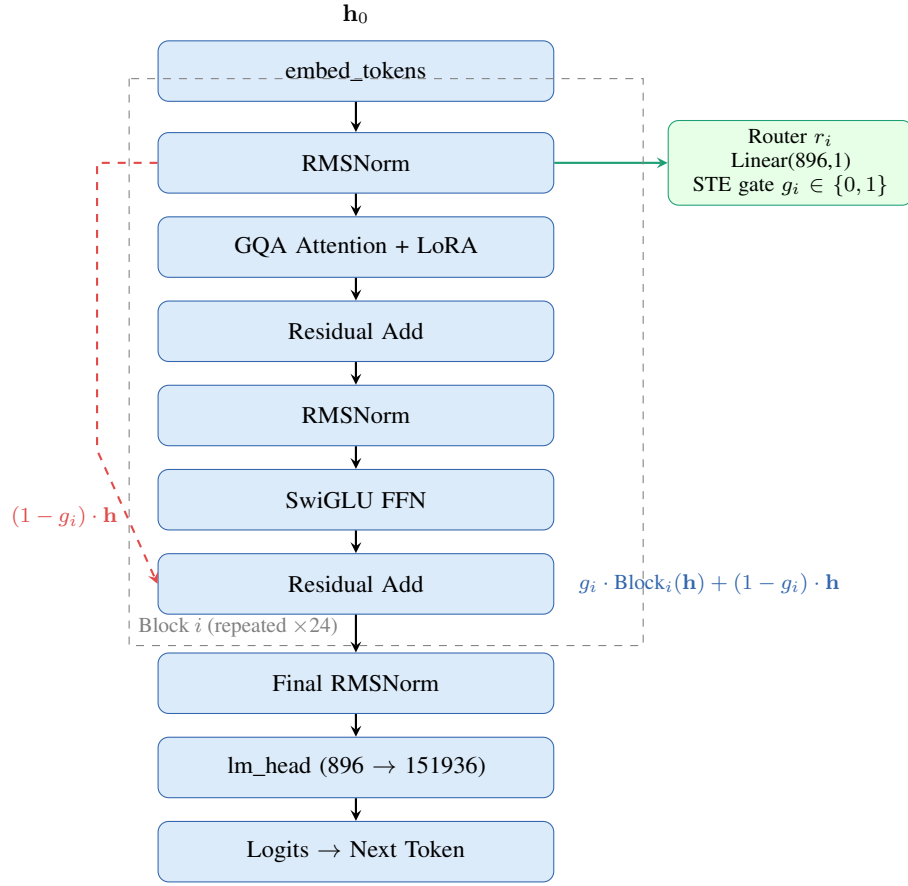


Figure 2: LayerRoute architecture. Each of 24 transformer blocks is augmented with a per-layer router (green) that computes a hard STE gate. When $g_i = 1$ the block runs with LoRA-adapted weights; when $g_i = 0$ the hidden state bypasses the block via the skip connection (dashed red).

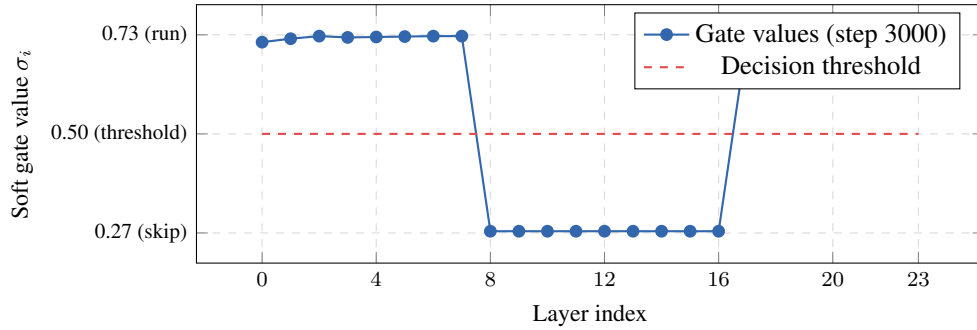


Figure 3: Learned gate values at step 3,000. Layers 0–7 and 17–23 stabilise at ≈ 0.73 (hard gate = 1, always run). Layers 8–16 stabilise at ≈ 0.27 (hard gate = 0, always skip). Gate variance: 0.050.

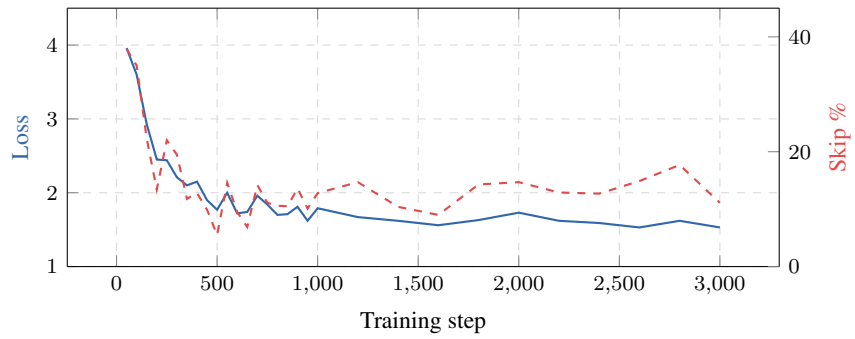


Figure 4: Training curves. Blue (left axis): LM loss converges from 3.96 to 1.53. Red dashed (right axis): skip percentage starts at 37.9% (biased init), falls as LoRA adapts, then stabilises at 10–18%.